



LV06 cstring, 재귀함수 호출의 깊이와 너비

<https://youtu.be/X-gpXreMejQ?feature=shared>

<https://youtu.be/5oLjqJQJ-PU?feature=shared>

C++ C-string과 고급 재귀함수

C-string 라이브러리

C-string의 개념과 중요성

문장을 비교 할 때 기존의 방법은 반복문을 사용해 두 배열의 글자가 다른지 확인해주는 작업을 거쳐야 했었다. 이제 여러 문자열을 처리 하기 위해서 직접 로직을 작성하지 않고 이미 만들어진 로직을 가져다 사용할수 있다.

기존 문자열 비교 방법 (수동)

```
#include <iostream>
using namespace std;

int flag = 0;
for (size_t i = 0; i < 10; i++)
{
    if (nameA[i] != nameB[i])
    {
        flag = 1;
        break;
    }
}
```

```

    }
}

if (flag == 0)
{
    //같다
}
else
{
    //다르다
}

```

기존 방법의 단점:

- 매번 반복문을 작성해야 함
- 문자열 길이를 미리 알아야 함
- 코드가 길고 복잡함
- 실수할 가능성이 높음

C-string 라이브러리 사용

이제는 **cstring** 라이브러리를 사용해서 두 문장이 같은지 비교해주면 된다.

```

#include <iostream>
#include <cstring> // C-string 라이브러리 포함
using namespace std;

int main()
{
    char strA[256] = "Hello";
    char strB[256] = "Hello";

    if (strcmp(strA, strB) == 0)
    {
        cout << "문자열이 같습니다." << endl;
    }
    else
    {
        cout << "문자열이 다릅니다." << endl;
    }
}

```

```

    }

    return 0;
}

```

주요 C-string 함수들

1. 문자열 길이 구하는 함수

```

// 문자열 길이 구하는 함수
int len = strlen(nameA);

```

2. 문자열 복사 함수 (복사할 위치, 복사할 소스)

```

// 문자열 복사 함수 (복사할 위치, 복사할 소스)
strcpy_s(nameC, nameA);

```

주요 C-string 함수 정리:

함수	기능	사용법
<code>strlen(str)</code>	문자열 길이 반환	<code>int len = strlen(str);</code>
<code>strcmp(str1, str2)</code>	문자열 비교	<code>if (strcmp(str1, str2) == 0)</code>
<code>strcpy(dest, src)</code>	문자열 복사	<code>strcpy(dest, src);</code>
<code>strcpy_s(dest, size, src)</code>	안전한 문자열 복사	<code>strcpy_s(dest, 256, src);</code>
<code>strcat(dest, src)</code>	문자열 연결	<code>strcat(dest, src);</code>
<code>strcat_s(dest, size, src)</code>	안전한 문자열 연결	<code>strcat_s(dest, 256, src);</code>

strcmp 함수 반환값:

- `0`: 두 문자열이 같음
- `양수`: 첫 번째 문자열이 사전순으로 뒤에 옴
- `음수`: 첫 번째 문자열이 사전순으로 앞에 옴

재귀함수의 깊이와 너비

재귀함수 호출 빈도(깊이)를 조절하는 방법

우리는 이전에 재귀함수의 호출 빈도(깊이)를 조절하는 방법을 배웠었다.

```

#include <iostream>
using namespace std;

void BBQ(int level)
{
    if (level == 3) // 종료 조건
    {
        return;
    }

    BBQ(level + 1); // 재귀 호출
}

int main()
{
    BBQ(0); // level 0부터 시작

    return 0;
}

```

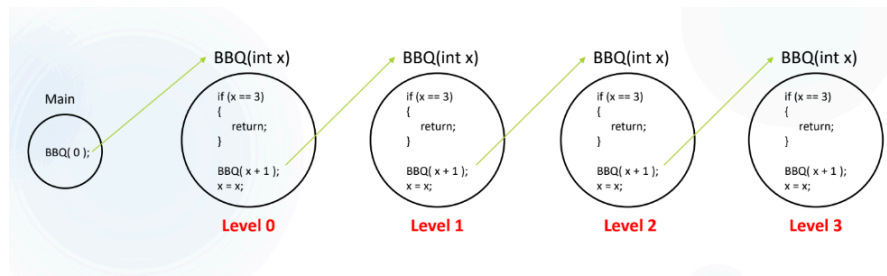
재귀 호출 과정:

Main → BBQ(0) → BBQ(1) → BBQ(2) → BBQ(3) → return

레벨별 스택 프레임:

- **Level 0:** BBQ(0) 호출, level=0
- **Level 1:** BBQ(1) 호출, level=1
- **Level 2:** BBQ(2) 호출, level=2
- **Level 3:** BBQ(3) 호출, level=3, return 실행

재귀함수 실행 과정 시각화



이번 재귀함수 함번이 아닌 두번을 호출한다면 어떻게 되는지 저장보자

트리 구조 재귀 호출

```
#include <iostream>
using namespace std;

void BBQ(int level)
{
    if (level == 2) // 깊이 2에서 종료
    {
        return;
    }

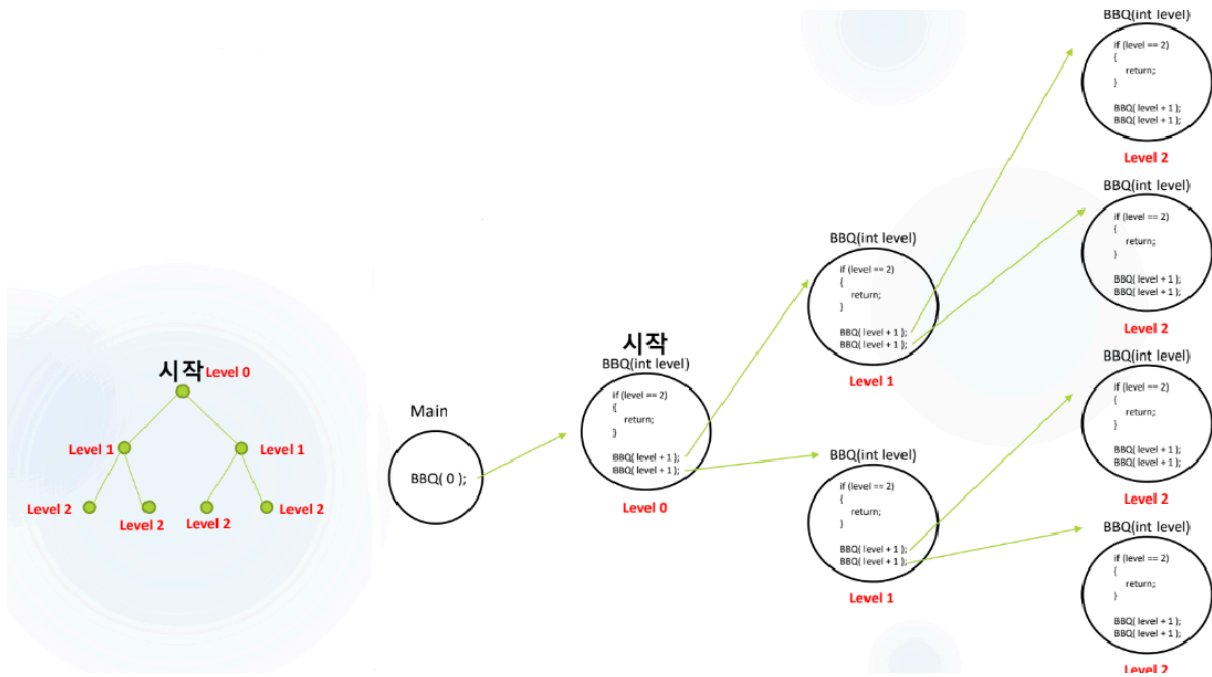
    BBQ(level + 1); // 첫 번째 재귀 호출
    BBQ(level + 1); // 두 번째 재귀 호출

    cout << "BBQ" << endl;
    //트리 형태로는 다음과 같이
    //for문으로도 표현 가능하다
    //for (size_t i = 0; i < 2; i++)
    //{
    //    BBQ(level + 1);
    //}
}

int main()
{
    BBQ(0);
}
```

```
return 0;
}
```

트리 구조 호출 과정 시각화



```

시작
BBQ(0)
 /  \
BBQ(1) BBQ(1)
 /  \  /  \
BBQ(2) BBQ(2) BBQ(2) BBQ(2)
 |    |    |    |
return return return return

```

호출 순서와 실행 흐름:

1. BBQ(0) 시작
2. 첫 번째 BBQ(1) 호출
3. 첫 번째 BBQ(2) 호출 → return
4. 두 번째 BBQ(2) 호출 → return
5. 첫 번째 BBQ(1) 완료, "BBQ" 출력

6. 두 번째 BBQ(1) 호출
7. 세 번째 BBQ(2) 호출 → return
8. 네 번째 BBQ(2) 호출 → return
9. 두 번째 BBQ(1) 완료, "BBQ" 출력
10. BBQ(0) 완료, "BBQ" 출력

최종 출력: "BBQ" 3번 출력

재귀 함수를 여러번 호출하게 되면 위와같은 형태로 출력되게 된다.

이를 잘 활용해서 여러가지 문제를 풀어보도록 하자.

만약 재귀함수 호출이 3개라면?

3개의 재귀 호출

```
#include <iostream>
using namespace std;

void BBQ(int level)
{
    if (level == 2)
    {
        return;

        //BBQ(level + 1);
        //BBQ(level + 1);
        //BBQ(level + 1);
        for (size_t i = 0; i < 3; i++)
        {
            BBQ(level + 1);
        }

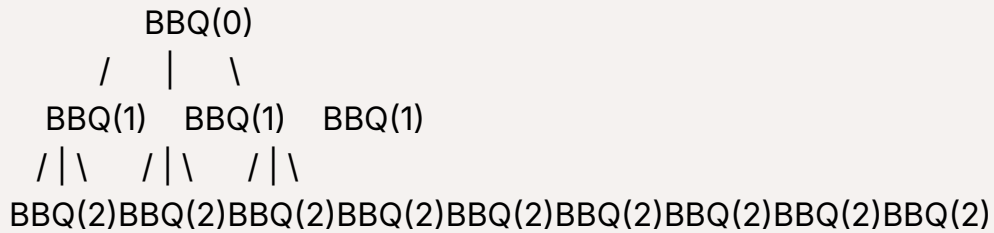
        cout << "BBQ" << endl;
    }

    int main()
    {
```

```
BBQ(0);

return 0;
}
```

3개 호출의 트리 구조



호출 횟수 계산:

- **Level 0:** 1회 (BBQ(0))
- **Level 1:** 3회 (BBQ(0)에서 3번 호출)
- **Level 2:** 9회 (각 BBQ(1)에서 3번씩 호출)
- **총 호출 횟수:** $1 + 3 + 9 = 13$ 회
- **"BBQ" 출력 횟수:** 4회 (3개의 BBQ(1) + 1개의 BBQ(0))

재귀함수 활용 예제

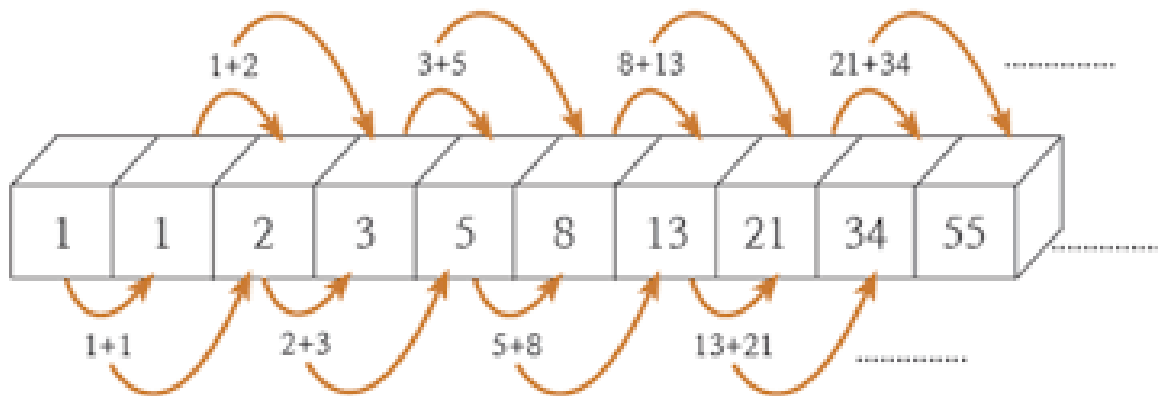
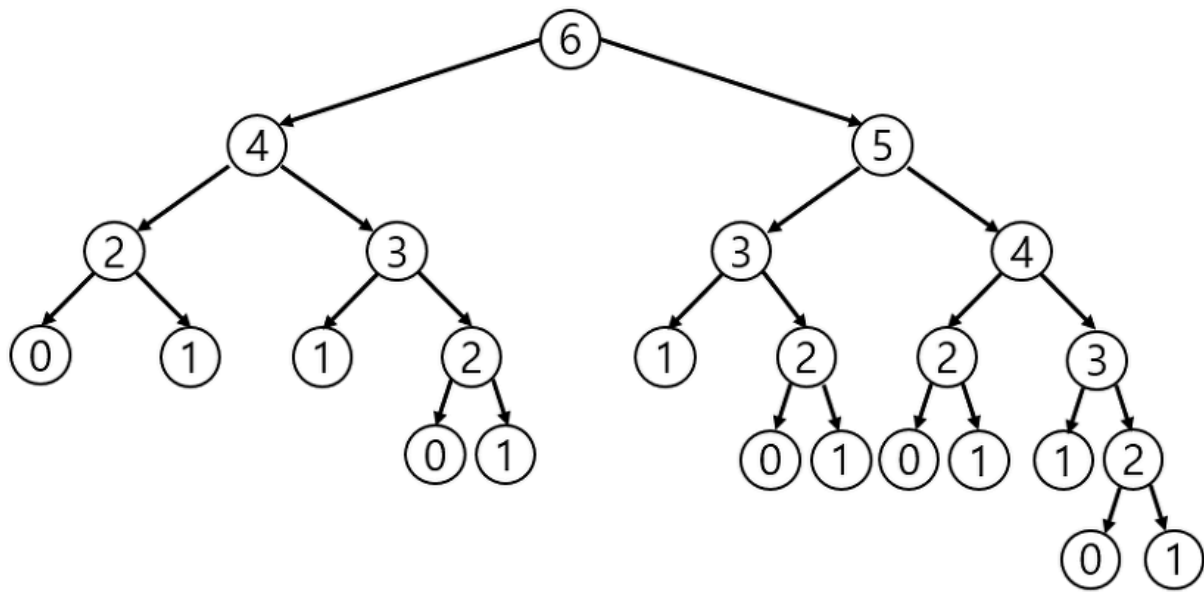
1. 팩토리얼 계산

```
int factorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }

    return n * factorial(n - 1);
}

// factorial(5) = 5 * 4 * 3 * 2 * 1 = 120
```


2. 피보나치 수열 (이진 트리 재귀)



```
int fibonacci(int n)
{
    if (n <= 1)
    {
        return n;
    }

    return fibonacci(n - 1) + fibonacci(n - 2); // 2개의 재귀 호출
}

int fibonacci(int n)
{
```

```

    if (n <= 1)
    {
        return n;
    }

    int left = fibonacci(n - 1); // 1개의 재귀 호출
    int right = fibonacci(n - 2); // 1개의 재귀 호출

    return left + right; // 2개의 재귀 호출
}

// fibonacci(4) 호출 트리:
//      fib(4)
//     /   \
//  fib(3)  fib(2)
//  /  \   /  \
// fib(2) fib(1) fib(1) fib(0)
// /  \
// fib(1) fib(0)

```

재귀함수 최적화와 주의사항

실무에서의 재귀 활용

파일 시스템 탐색

```

void listDirectory(const string& path, int depth = 0)
{
    // 들여쓰기 출력
    for (int i = 0; i < depth; i++) cout << " ";
    cout << path << endl;

    // 하위 디렉토리들에 대해 재귀 호출
    // for (각 하위 디렉토리)
    // {
    //     listDirectory(하위디렉토리, depth + 1);
    // }
}

```

```
// }  
}
```

JSON 파싱

```
void parseJSON(JSONNode node, int depth = 0)  
{  
    if (node.isObject())  
    {  
        for (auto& child : node.children)  
        {  
            parseJSON(child, depth + 1); // 재귀적으로 파싱  
        }  
    }  
}
```

C-string 라이브러리는 문자열 처리를 간편하게 해주며, 다중 재귀는 복잡한 구조를 표현하는 데 유용합니다. 하지만 성능과 메모리 사용량을 항상 고려해야 하며, 필요에 따라 메모이제이션이나 반복문으로 최적화하는 것이 중요합니다.

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 손이 안 움직이고,
- 복습을 하려 해도 무엇을 다시 봐야 할지 모르겠고,
- 질문할 곳도 없고,
- 유튜브는 결국 정답을 따라 치는 것밖에 안 되는 것 같고.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 암암코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얌얌코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 **스스로 쪼개고 설계하는 힘**
- 다양한 조건을 만족시키는 **실제 구현 능력**
- 기능 단위가 아닌, **프로그램 단위로 사고하는 습관**
- 마침내 **자신의 힘으로 코드를 끝까지 작성하는 경험**

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 **훈련**,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얌얌코딩 상담방](#)

▼ oopy:hide

연습문제

그 동안 문장을 다루는데 큰 불편함을 느꼈을 것입니다.

non-library 단계에서 library를 사용하는 방향으로 넘어가는 시점입니다.

cstring library를 사용해서 편리하게 문장을 다뤄 봅시다.

그리고 재귀호출 두번째 수업인 Branch와 Level에 대해 이해하고, 훈련 해 봅시다.

재귀호출은 어렵지 않습니다.

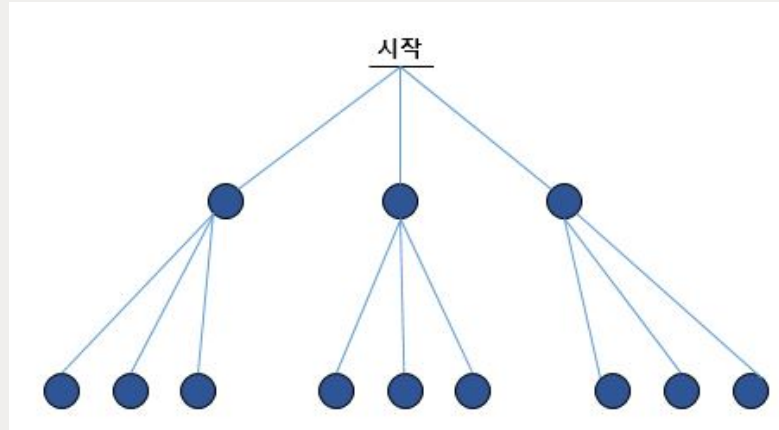


재귀호출이 3개일때

문제 1번

그림과 같이 동작되는 프로그램을 작성하세요.

(입출력 값이 없는 문제입니다, 소스코드만 작성 후 제출 해 주세요)



로그인 처리하기

문제 2번

운영자의 ID와 PASSWORD는

"qlqlaqkq"

"tkaruqtkf" 입니다. (비빔밥 / 삼겹살)

ID와 PASSWORD를 입력받고,

운영자 ID와 PASSWORD가 정확히 일치한다면, "LOGIN"을 출력 해 주세요

아니면 "INVALID"를 출력 해 주세요

(cstring library를 사용 해 주세요)

입력 예제

qlqlaqkq

tkaruqtkf

출력 결과

LOGIN



Level과 Branch

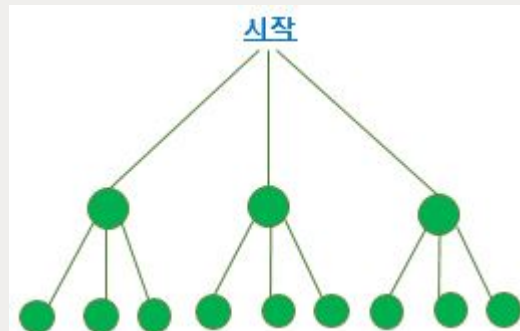
문제 3번

level과 branch를 입력 받고 그만큼 재귀호출을 해 주세요
(출력결과 없음)

ex)

level = 2

branch = 3



입력 예제

2

3



입력받은 Level까지 재귀함수 동작

문제 4번

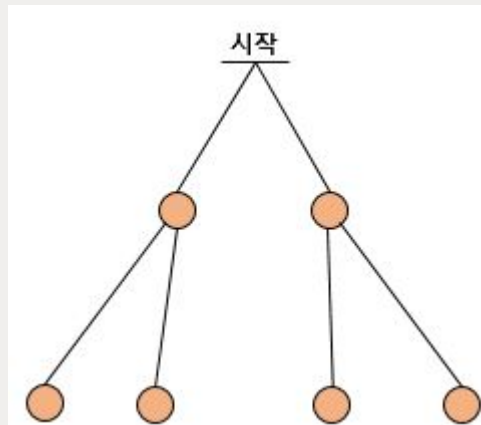
숫자 1개를 입력받고,

입력받은 Level 까지 재귀함수가 동작되도록 코딩해주세요

함수가 시작될 때 Level을 출력하면 됩니다.

ex1)

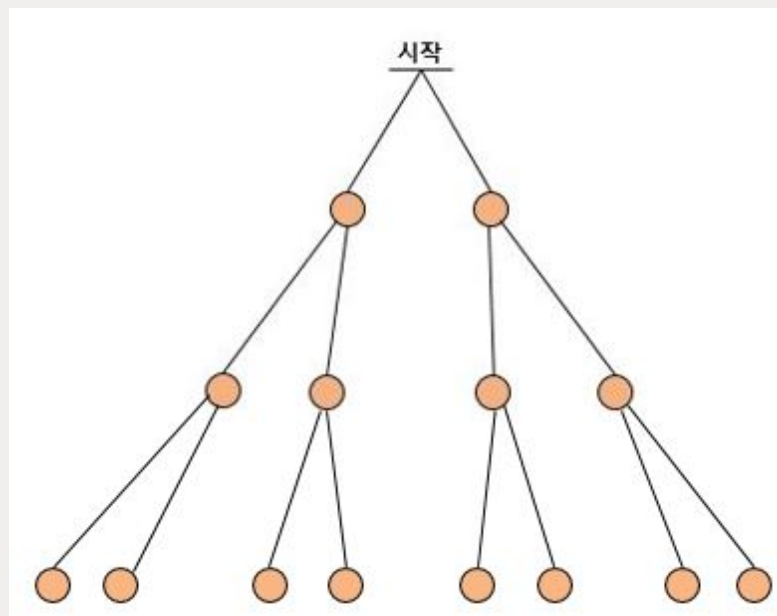
입력 : 2



출력결과 : 0122122

ex2)

입력 : 3



출력결과 : 012332331233233

입력 예제

2

출력 결과

0122122



긴문장을 맨 앞으로

문제 5번

세 문장을 2차원 배열에 입력 받으세요.(최대 10글자)

가장 긴문장과 첫 번째로 입력 받은 문장을 **SWAP** 한 후,
배열에 있는 문장을 모두 출력 해주세요.

(strlen과 strcpy를 사용하면 됩니다)

입력 예제

KFC

ABCDEFGFG

BBQ

출력 결과

ABCDEFGFG

KFC

BBQ



문제 6번

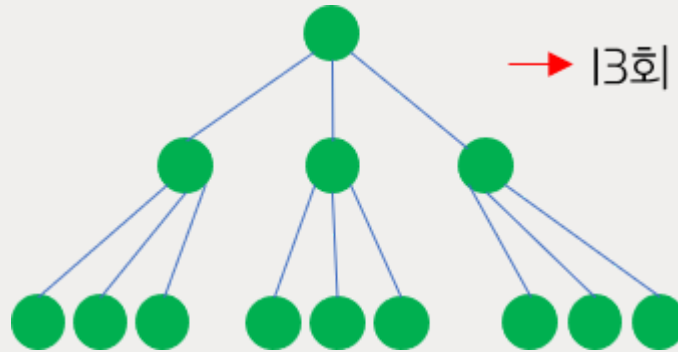
branch와 **Level**을 입력받고 재귀 함수가 호출되는 횟수를 **counting**해서 출력해주세요.

예로들어 3 2 을 입력받으면 재귀함수는 총 13회 호출 됩니다.

ex)

3 2

(branch) (level)



[힌트]

전역변수를 두고 함수를 호출할 때마다 **counting** 해주면 됩니다.

입력 예제

4 3

출력 결과

85



글자수만큼 손가락 접기

문제 7번

한문장을 입력받고 글자수를 구해주세요.

그리고 **재귀호출을 이용**해 다음과 같이 출력 해주세요.

만약 **ABCDE**를 입력받았다면 **5글자** 이므로 **5 4 3 2 1 2 3 4 5** 출력

만약 **BBQ**를 입력받았다면 **3글자** 이므로 **3 2 1 2 3** 을 출력 하면 됩니다.

(문장의 내용은 중요하지 않습니다. 글자수가 중요합니다.)

입력 예제

ABCDE

출력 결과

5 4 3 2 1 2 3 4 5



생일선물 마우스

문제 8번

생일선물로 마우스를 직접 만들고,

내 연인에게 선물을 주며 고백하려 합니다.

마우스는 현재 $(y,x)=(5,5)$ 에 위치합니다.

up / down / left / right / click 명령어들이 존재합니다.

명령어 수(n)와 명령어들을 입력받고 명령어 대로 수행 해주세요.

[명령어]

up : y축으로 한칸 -1

down : y축으로 한칸 +1

left : x축으로 한칸 -1

right : x축으로 한칸 +1

click : 현재 좌표 출력

[입력예제]

4 //명령어의 수

up

click //출력

right

click //출력

[출력결과]

4,5

4,6

입력 예제

3

down

down

click

출력 결과

7,5

복습문제



문제 1번

D	T	K
E	A	P
C	Q	G
P	H	B

4 × 3 배열에 A, B가 하나씩 적혀 있습니다.

A와 B를 찾아 상하좌우로 몇칸 떨어져 있는지 출력하면 됩니다.

A위치에서 오른쪽 한칸, 밑으로 두칸 움직이면 됩니다.

답은 총 세칸 입니다.

입력 예제

DTK

EAP

CQG

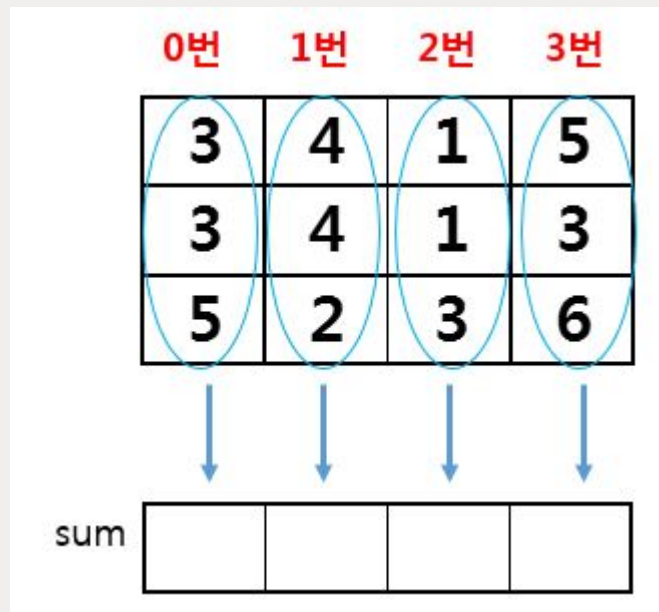
PHB

출력 결과

3



문제 2번



각 세로줄의 합을 구해서 sum이라는 배열에 넣어주세요.

그리고 index라는 변수에 숫자 하나를 입력받고,

sum[index] 값을 출력 해주세요.

입력 예제

2

출력 결과

5



문제 3번

문장 하나와 문자 2개를 입력받아주세요

문장에서, 문자가 존재하는 곳 좌우의 값을 '#'으로 바꾸어 출력 해 주세요

- 입력받은 문자는 문장에 각각 1개씩만 존재합니다.
- 예제3와 같이, #을 넣는게 불가능하다면 넣지 않습니다.
- 설계를 꼼꼼히 하신 후 풀어야 합니다.

예제1) **APKDB**를 입력받고 **P D**를 입력받으면 **#P#D#**을 출력하면 됩니다.

예제2) **REWUQ**를 입력받고 **W U**를 입력받으면 **R####**이 출력되면 됩니다

예제3) **ABCDEFGH**를 입력받고, **A H**를 입력받으면 **A#CDE#G**를 출력해야 합니다.

입력 예제

APKDB

P D

출력 결과

#P#D#



문제 4번

우주선인 2차배열에 문자들이 둥둥 공중에 떠있습니다.

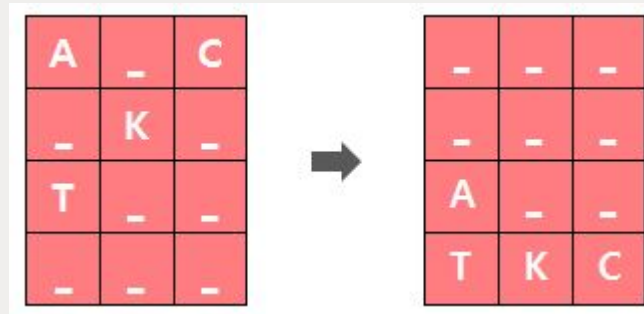
소행성과의 충돌 위험으로 중력이 있는 행성에 불시착하였더니

문자들이 바닥에 떨어졌습니다.

아래 그림처럼 입력하고 중력을 받아 바닥에 떨어진 문자상태를 출력하면 됩니다.

[입력]

[출력]



입력 예제

A_C

K

T__

출력 결과

A__

TKC



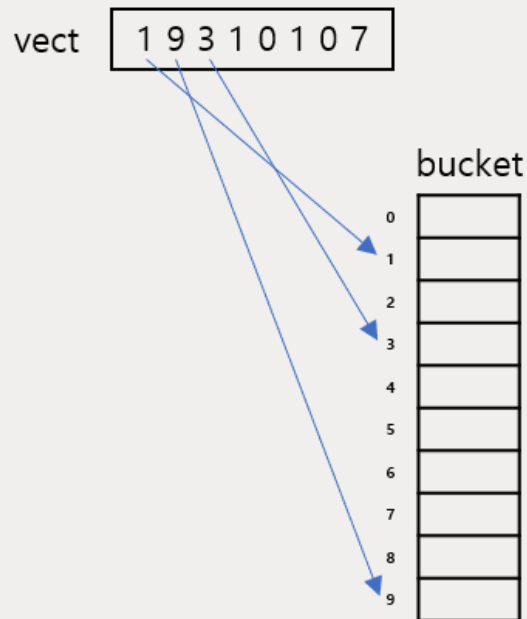
문제 5번

숫자 8개(0 ~ 9)를 입력받습니다.

입력받은 숫자들을 아래의 규칙을 이용해서 정렬 후 출력 해 주세요.

[규칙]

먼저, 입력받은 vect배열 값들을 하나씩 탐색하면서 bucket 배열에 counting 합니다.



이렇게 counting 후 만들어지는 배열은 다음과 같습니다.

bucket	
0	2
1	3
2	0
3	1
4	0
5	0
6	0
7	1
8	0
9	1

이제 bucket 배열을 이용해서 정렬된 결과를 출력하면 됩니다.

위 예제에서

숫자 0은 2회 발견되었으니 '0' 2회 출력

숫자 1은 3회 발견되었으니 '1' 3회 출력

숫자 3은 1회 발견되었으니 '3' 1회 출력

...

숫자 9는 1회 발견되었으니 '9' 1회 출력

따라서 출력되는 결과는 다음과 같습니다.

00111379

입력 예제

19310107

출력 결과

00111379



문제 6번

아래 배열을 하드코딩 해 주세요

1	5	3
4	5	5
3	3	5
4	6	2

숫자 두개를 변수 a, b에 입력 받으세요.

a ~ b 사이에 있는 숫자들을 모두 0으로 바꿔 주세요. ($a \leq x \leq b$)

그리고 출력할때 0인 부분만 #으로 바꿔 출력해주세요.

입력 예제

3 4

출력 결과

1 5 #

5 5

5

6 2



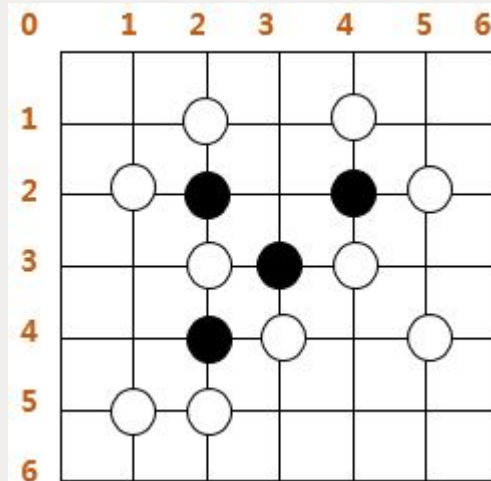
바둑이 게임

문제 7번

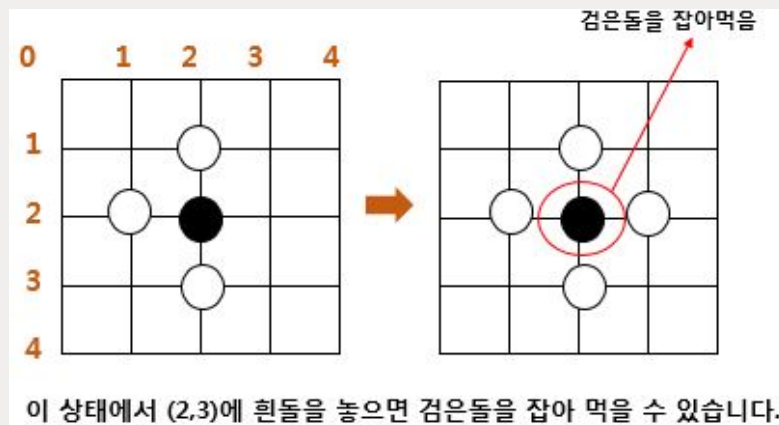
바둑이판 상태를 하드코딩 해주세요.

흰돌을 놓을 곳(좌표)을 입력 받으세요.

입력받은 좌표에 흰돌을 놓을때, 돌 몇개를 잡아먹을 수 있는지 출력 하세요.



위, 아래, 왼쪽, 오른쪽을 감싸고 있을때만 안쪽에 있는 돌을 잡아 먹을 수 있습니다.



- 이 게임은 바둑과는 다릅니다.

검을돌을 잡아먹을 수 있는 방법은 오로지 위, 아래, 왼쪽, 오른쪽을 감쌀때만 입니다.

- `direct`를 써서 풀어주세요

입력 예제

2 3

출력 결과

3



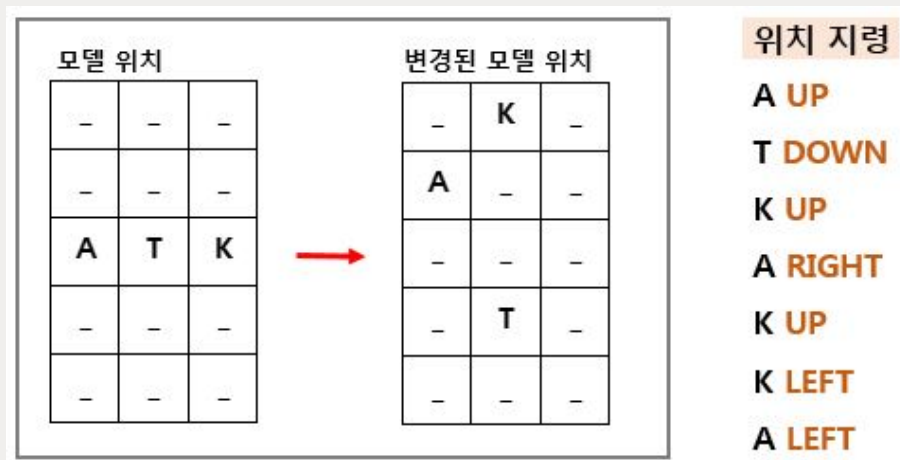
문제 8번

모델들이 한줄로 무대위에 서있습니다.

디렉터는 모델들에게 **지시를 7번** 합니다.

지시 이후 모델들의 위치를 출력 해주세요.

(굉장한 노가다 문제입니다)



입력 예제(char findChar, char* direction)

A UP
T DOWN
K UP
A RIGHT
K UP
K LEFT
A LEFT

출력 결과

K
A_
_
T
_

